Inserting Fmap Calls with Coercive Subtyping

This paper shows how to use subtyping to infer uses of the basic monad operations.

CCS Concepts: • **Software and its engineering → Functional languages**.

Additional Key Words and Phrases: subtyping, functors

ACM Reference Format:
. 2026. Inserting Fmap Calls with Coercive Subtyping. *J. ACM* 37, 4, Article 111 (August 2026), 2 pages. https://doi.org/XXXXXXX.XXXXXXX

1 Introduction

(1) In Haskell, effects are modeled by functors T A equipped with useful typeclass instances like Applicative or Monad.
(2) Across modules, the functor in use varies with the collection of necessary effects.
(3) A front-end that does I/O will need to use a functor that supports IO, but pure computations might not use a functor at all.
(4) At a boundary between two modules, calls need to be decorated with liftings between their respective functors; if no such lifting exists then the calling module may need to be reorganized around a new functor that subsumes the old ones.
(5) While altering the types in use requires human input, inserting lift operations seems mechanical.
(6) In this paper, we automate the insertion of fmap lifts with a subtyping discipline that makes $A \to B$ a subtype of $FA \to FB$.
(7) The subtyping judgement is *coercive*: each subtyping $A \leq B$ comes with a function $c : A \to B$ that is inserted during type checking.
(8) Despite the power of the fmap rule, the subtyping judgement remains coherent: for a given A and B, all coercions $c : A \leq B$ are equivalent.
(9) Our proof, checked in Agda, is a normalization proof for coercions that avoids the use of a small-step reduction relation.
(10) Instead, it achieves normalization by evaluation: coercions are mapped to a domain of normal forms, which can be reified back into coercions.
(11) The system of coercions in normal form $n : A \leq B$ can also serve as an algorithmic subtyping judgement: it admits transitivity and reflexivity, but has unique derivations.
(12) Besides giving some confidence in the safety of this programming model, the proof of soundness has independent interest as a case study in normalization by evaluation and it makes good use of Agda's dependent pattern matching features.

Author's Contact Information:

Permission to make digital or hard copies of all or part of this work for personal or classroom use is granted without fee provided that copies are not made or distributed for profit or commercial advantage and that copies bear this notice and the full citation on the first page. Copyrights for components of this work owned by others than the author(s) must be honored. Abstracting with credit is permitted. To copy otherwise, or republish, to post on servers or to redistribute to lists, requires prior specific permission and/or a fee. Request permissions from permissions@acm.org.
© 2026 Copyright held by the owner/author(s). Publication rights licensed to ACM.
ACM 1557-735X/2026/8-ART111
https://doi.org/XXXXXXX.XXXXXXX

2 Background

Some examples

A discussion of how incoherent subtyping happens / why it could happen

3 Type System

Grammar of c

4 Normalized Coercions

Grammar of n, ne

5 Soundness and Completeness

Discussion of the NbE method.

Discussion of the proof.

6 Implementation

Using Chronolog, we did some examples.

7 Related Work

Reynolds?

8 Conclusion

References